Esraa Ashraf

TASK 3

1-

```
module dff_ep_tb2();
    reg sys_clk, pre_n, din, en;
    wire dout;
    reg dout_temp;
    integer loop_i = 0;

    dff_en_pre U1(en, din, dout, sys_clk, pre_n);

    initial begin
    sys_clk = 0;
    forever
    #1 sys_clk = ~sys_clk;
    end

    initial begin
    pre_n = 0;
    din = 0;
    en = 0;
    @(negedge sys_clk);
    pre_n = 1;
    for (loop_i = 0; loop_i < 10000; loop_i = loop_i + 1) begin
    din = $random;
    @(negedge sys_clk);
    if (dout != 1'b1) begin
    $display("Err - FF out wrong");
    $stop;
    end
    end
    en = 1;
    for (loop_i = 0; loop_i < 80000; loop_i = loop_i + 1) begin
    din = $random;
    @(negedge sys_clk);
    if (dout != din) begin
    $display("Err - FF out wrong");
    $stop;
    end
    end
    for (loop_i = 0; loop_i < 10000; loop_i = loop_i + 1) begin
    dout_temp = dout;
```

```
    din = $random;
    en = $random;
    @(negedge sys_clk);
    if ((en == 1 && (dout != din)) || (en == 0 && (dout != dout_temp))) begin
    $display("Err - FF out wrong");
    $stop;
    end
    end
    end
endmodule
```

2-

```
module tb_dff_check();

    reg d_pin, rst_pin, clk_pin;
    wire q_inst, q_gold, qb_inst, qb_gold;

    d_t_ff #("DFF") FF1(d_pin, q_inst, qb_inst, rst_pin, clk_pin);
    dff_rstn GOLDEN(d_pin, q_gold, qb_gold, rst_pin, clk_pin);

    integer cnt;

    initial begin
    clk_pin = 0;
    forever
    #1 clk_pin = ~clk_pin;
    end

    initial begin
    rst_pin = 0;
    d_pin = 0;
    @(negedge clk_pin);
    rst_pin = 1;
    for (cnt = 0; cnt < 10000; cnt = cnt + 1) begin
    d_pin = $random;
    @(negedge clk_pin);
    if ((q_gold != q_inst) || (qb_gold != qb_inst)) begin
    $display ("FAIL");
    $stop;
    end
    end
    $stop;
    end
```

```
    initial begin
    $monitor("d=%b, rst=%b, clk=%b, q_g=%b, q_i=%b", d_pin, rst_pin, clk_pin,
q_gold, qb_inst);
    end
endmodule
```

```
module tff_verify();
reg t_in, rst_in, clock_in;
wire q_dut, q_exp, qb_dut, qb_exp;

d_t_ff #("TFF") DUT(t_in, q_dut, qb_dut, rst_in, clock_in);
tff EXPECTED(t_in, q_exp, qb_exp, rst_in, clock_in);

integer idx;

initial begin
    clock_in = 0;
    forever
    #1 clock_in = ~clock_in;
end

initial begin
    rst_in = 0;
    t_in = 0;
    @(negedge clock_in);
    rst_in = 1;
    for (idx = 0; idx < 10000; idx = idx + 1) begin
    t_in = $random;
    @(negedge clock_in);
    if ((q_exp != q_dut) || (qb_exp != qb_dut)) begin
    $display ("MISMATCH");
    $stop;
    end
    end
    $stop;
end

initial begin
    $monitor("t=%b, rst=%b, clk=%b, q_exp=%b, q_dut=%b, qb_exp=%b", t_in, rst_in,
clock_in, q_exp, q_dut, qb_exp);
end
endmodule
```

3-

```
 module bcd_counter_v1(clk_i, rst_i, clk_div_o);

input clk_i, rst_i;
output reg clk_div_o;

reg [3:0] cnt;

always @(posedge clk_i or posedge rst_i) begin
    if (rst_i)
    cnt <= 4'b0000;
    else begin
    if (cnt == 4'b1001) begin
    cnt <= 4'b0000;
    end else begin
    cnt <= cnt + 1;
    end
    end
end

always @(posedge clk_i or posedge rst_i) begin
    if (rst_i)
    clk_div_o <= 1'b0;
    else begin
    if (cnt < 4'b0101)
    clk_div_o <= 0;
    else
    clk_div_o <= 1;
    end
    end
endmodule
```

```
module bcd_tb2();
reg clk_s, rst_s;
wire clk_div_s;

bcd_counter_v1 UUT(clk_s, rst_s, clk_div_s);

initial begin
    clk_s = 0;
    forever
    #1 clk_s = ~clk_s;
end

initial begin
    rst_s = 1;
    @(negedge clk_s);
    rst_s = 0;
    repeat(100) @(negedge clk_s);
    $stop;
end

initial begin
    $monitor("div=%b, rst=%b, clk=%b", clk_div_s, rst_s, clk_s);
end
endmodule
```

4-

```
 module ripple_cnt_struct (clk_in, rst_n, cnt_out);


input clk_in, rst_n;
output [3:0] cnt_out;

wire w_q0, w_qn0, w_q1, w_qn1, w_q2, w_qn2, w_q3, w_qn3;

dff_rstn    ff0 (.d (w_qn0),
    .clk (clk_in),
    .rstn (rst_n),
    .q (w_q0),
    .qn (w_qn0));

dff_rstn    ff1 (.d (w_qn1),
    .clk (w_q0),
    .rstn (rst_n),
```

```
    .q (w_q1),
    .qn (w_qn1));

dff_rstn    ff2 (.d (w_qn2),
    .clk (w_q1),
    .rstn (rst_n),
    .q (w_q2),
    .qn (w_qn2));

dff_rstn    ff3 (.d (w_qn3),
    .clk (w_q2),
    .rstn (rst_n),
    .q (w_q3),
    .qn (w_qn3));

assign cnt_out = {w_qn3, w_qn2, w_qn1, w_qn0};

endmodule
```

```
module tb_ripple_cnt();
reg rst, clk;
wire [3:0] cnt;

ripple_cnt_struct DUT(clk, rst, cnt);

initial begin
    clk = 0;
    forever
    #1 clk = ~clk;
end

initial begin
    rst = 0;
    @(negedge clk);
    rst = 1;
    repeat (1000) @(negedge clk);
    $finish;
end

initial begin
    $monitor("rst=%b, clk=%b, cnt=%b", rst, clk, cnt);
end
endmodule
```

5-

```
 module shifter (sclr_pin, sset_pin, sin_pin, ld_pin, d_bus, ck_pin, en_pin,
aclr_pin, aset_pin, sout_pin, q_bus);

    parameter LOAD_A = 1;
    parameter LOAD_S = 1;
    parameter DIR = "LEFT";
    parameter W = 8;

    input sclr_pin, sset_pin, sin_pin, ld_pin, ck_pin, en_pin, aclr_pin,
aset_pin;
    input [W-1:0] d_bus;
    output reg sout_pin;
    output reg [W-1:0] q_bus;

    always @(posedge ck_pin or posedge aclr_pin or posedge aset_pin) begin
    if (aclr_pin)
    q_bus <= {W{1'b0}};
    else if (aset_pin)
    q_bus <= LOAD_A;
    else begin
    if (en_pin) begin
    if (sclr_pin)
    q_bus <= {W{1'b0}};
    else if (sset_pin)
    q_bus <= LOAD_S;
    else if (ld_pin)
    q_bus <= d_bus;
    else begin
    if (DIR == "LEFT")
    {sout_pin, q_bus} <= {q_bus, sin_pin};
    else if (DIR == "RIGHT")
    {q_bus, sout_pin} <= {sin_pin, q_bus};
    end
    end
    end
    end
endmodule
```

```
module shifter_tb2();
parameter P_LOAD_A = 2;
parameter P_LOAD_S = 4;
parameter P_DIR = "LEFT";
parameter P_W = 8;

reg sclr, sset, sin, ld, clk, en, aclr, aset;
reg [P_W-1:0] data_bus;
wire sout;
wire [P_W-1:0] q_bus;

shifter #(P_LOAD_A, P_LOAD_S, P_DIR, P_W) U1(sclr, sset, sin, ld, clk, en, aclr,
aset, data_bus, sout, q_bus);

initial begin
    clk = 0;
    forever
    #1 clk = ~clk;
end

initial begin
    aclr = 1;
    aset = 1;
    repeat(20) begin
    sclr = $random;
    sset = $random;
    ld = $random;
    en = $random;
    data_bus = $random;
    sin = $random;
    @(negedge clk);
    if (q_bus != 0) begin
    $display("Async FAIL - ACLR");
    $stop;
    end
    end
end

aclr = 0;
repeat(20) begin
    sclr = $random;
    sset = $random;
    ld = $random;
```

```
    en = $random;
    data_bus = $random;
    sin = $random;
    @(negedge clk);
    if (q_bus != P_LOAD_A) begin
    $display("Async FAIL - ASET");
    $stop;
    end
end

aset = 0;
sset = 1;
sclr = 1;
repeat(20) begin
    ld = $random;
    en = $random;
    data_bus = $random;
    sin = $random;
    @(negedge clk);
    if (q_bus != 0) begin
    $display("Sync FAIL - SCLR");
    $stop;
    end
end

sclr = 0;
repeat(20) begin
    ld = $random;
    en = $random;
    data_bus = $random;
    sin = $random;
    @(negedge clk);
    if (q_bus != P_LOAD_S) begin
    $display("Sync FAIL - SSET");
    $stop;
    end
end

ld = 1;
repeat(20) begin
    en = $random;
    data_bus = $random;
    sin = $random;
    @(negedge clk);
    if (q_bus != data_bus && en == 1) begin
```

```
    $display("Load FAIL");
    $stop;
    end
end

ld = 0;
en = 1;
repeat(1000) begin
    data_bus = $random;
    sin = $random;
    @(negedge clk);
end

en = 0;
repeat(200) begin
    data_bus = $random;
    sin = $random;
    @(negedge clk);
end

repeat(200) begin
    en = $random;
    data_bus = $random;
    sin = $random;
    @(negedge clk);
end

$stop;
end
endmodule
```

6-

```
module sle_cell(D, CLK, EN, ALN, ADN, SLN, SD, LAT, Q);

  input D, CLK, EN, ALN, ADN, SLN, SD, LAT;
  output Q;

  reg q_ff, q_lat;

  always @(posedge CLK or negedge ALN) begin
  if (~ALN)
  q_ff <= ~ADN;
  else if (EN) begin
  if (!SLN)
  q_ff <= SD;
  else
  q_ff <= D;
  end
  end

  always @(*) begin
  if (~ALN)
  q_lat <= ~ADN;
  else if (CLK) begin
  if (EN) begin
  if (!SLN)
  q_lat <= SD;
  else
  q_lat <= D;
  end
  end
  end

assign Q = (LAT)? q_lat : q_ff;

endmodule
```

```
module sle_test();
reg D, CLK, EN, ALN, ADN, SLN, SD, LAT;
wire Q;

sle_cell U1(D, CLK, EN, ALN, ADN, SLN, SD, LAT, Q);

initial begin
    CLK = 0;
    forever
    #1 CLK = ~CLK;
end

initial begin
    ALN = 0;
    EN = 0;
    ADN = 0;
    SLN = 0;
    SD = 0;
    LAT = 0;
    @(negedge CLK);
    ALN = 1;
    repeat (1000) begin
    EN = $random();
    ALN = $random();
    ADN = $random();
    SLN = $random();
    SD = $random();
    D = $random();
    @(negedge CLK);
    end

    LAT = 1;
    repeat (1000) begin
    EN = $random();
    ALN = $random();
    ADN = $random();
    SLN = $random();
    SD = $random();
```

```
    D = $random();
    @(negedge CLK);
    end
    $stop;
end

initial begin
    $monitor("D=%b, CLK=%b, EN=%b, ALN=%b, Q=%b", D, CLK, EN, ALN, Q);
end
endmodulemodule sle_test();
reg D, CLK, EN, ALN, ADN, SLN, SD, LAT;
wire Q;

sle_cell U1(D, CLK, EN, ALN, ADN, SLN, SD, LAT, Q);

initial begin
    CLK = 0;
    forever
    #1 CLK = ~CLK;
end

initial begin
    ALN = 0;
    EN = 0;
    ADN = 0;
    SLN = 0;
    SD = 0;
    LAT = 0;
    @(negedge CLK);
    ALN = 1;
    repeat (1000) begin
    EN = $random();
    ALN = $random();
    ADN = $random();
    SLN = $random();
    SD = $random();
    D = $random();
    @(negedge CLK);
    end

    LAT = 1;
    repeat (1000) begin
    EN = $random();
    ALN = $random();
    ADN = $random();
```

```
    SLN = $random();
    SD = $random();
    D = $random();
    @(negedge CLK);
    end
    $stop;
end

initial begin
    $monitor("D=%b, CLK=%b, EN=%b, ALN=%b, Q=%b", D, CLK, EN, ALN, Q);
end
endmodule
```